

Design Patterns in Python

*SOLID Principles, Interfaces, and the Gang of Four Patterns That Power
Production-Grade Code*

Builder · Prototype · Adapter · Proxy · Strategy

SECTION 1

Theory & Concepts

Chapter 1: What Are Design Patterns?

1.1 The Core Idea

Imagine every time you needed a door, you had to invent the concept of a hinge from scratch. That would be absurd — hinges are a solved problem.

Design patterns are the software equivalent of the hinge. They are reusable, time-tested solutions to problems that show up again and again when building software — not finished code you copy-paste, but a blueprint that you adapt to your specific situation.

A design pattern describes:

- The recurring problem it solves.
- A general structural and behavioral solution.
- Trade-offs and consequences of using it.

It deliberately stops short of being a complete, ready-to-run implementation — that part is always up to you, the developer, because every codebase is a little different.

1.2 Why Bother Learning Them?

You might be wondering: "Can't I just write code that works, without worrying about pattern names?" Technically yes — but here's what you gain by learning patterns properly:

- Reusable solutions — you stop reinventing the wheel for problems that have already been solved well.
- Readability — code organized around a known pattern is instantly recognizable to other developers.
- Faster development — less time spent designing from zero, more time spent solving your actual business problem.
- Maintainability — patterns tend to produce loosely coupled code that's easier to change later.
- A shared vocabulary — when you tell a teammate "I used the Strategy pattern here," they instantly understand the shape of your code without reading every line.

 **Think of It Like This:** Design patterns are not rules you must follow rigidly — they're a vocabulary of proven shapes. Once you recognize the shape of a problem, you can reach for the matching pattern instead of starting from a blank page.

1.3 A Bit of History

Design patterns as a formal discipline were popularized by the famous 1995 book

Design Patterns: Elements of Reusable Object-Oriented Software, written by four authors — Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. Because there were four of them, the software community affectionately nicknamed them the

"Gang of Four" (GoF). Their book remains the authoritative reference on design patterns even today, and almost every modern pattern catalogue — including this chapter — traces its roots back to it.

Chapter 2: The Three Families of Design Patterns

The GoF organized design patterns into three broad categories, based on what aspect of your software design they address. Understanding this classification first makes it much easier to remember individual patterns later — instead of memorizing 20+ patterns in isolation, you'll know which "drawer" each one belongs in.

Category	Focuses On	Examples
Creational	How objects get created	Builder, Prototype, Factory, Singleton
Structural	How classes/objects are composed	Adapter, Proxy, Decorator, Facade
Behavioral	How objects communicate & interact	Strategy, Observer, Command, State

2.1 Creational Patterns

These patterns deal with the mechanics of object creation. Instead of calling a constructor directly and hardcoding exactly how an object gets built, creational patterns hide that complexity behind a cleaner interface — improving flexibility and promoting loose coupling between the code that needs an object and the code that knows how to build it.

2.2 Structural Patterns

These patterns are concerned with how classes and objects are composed to form larger structures. They help you organize relationships between objects, simplify complex structures, and improve the overall flexibility of your system's architecture.

2.3 Behavioral Patterns

These patterns define how objects communicate and collaborate with one another at runtime. They manage responsibilities, reduce tight coupling between collaborating objects, and focus heavily on dynamic, runtime behavior rather than static structure.

Chapter 3: The SOLID Principles

Before diving into individual patterns, we need a shared foundation. The

SOLID principles are five rules of object-oriented design that underpin almost every design pattern you'll encounter. If design patterns are the "recipes," SOLID principles are the "cooking techniques" that make those recipes actually good.

SOLID is an acronym for five principles:

- S — Single Responsibility Principle
- O — Open/Closed Principle
- L — Liskov Substitution Principle
- I — Interface Segregation Principle
- D — Dependency Inversion Principle

3.1 Single Responsibility Principle (SRP)

The Rule: A class should have only one reason to change.

In plain English — give every class exactly one job. If a class is responsible for calculating an invoice AND printing it AND emailing it, you've bundled three unrelated responsibilities into one class. Change the printing logic, and you risk breaking the calculation logic too, even though they have nothing to do with each other.

```
class Invoice:
    def calculate_total(self):
        pass

class InvoicePrinter:
    def print_invoice(self):
        pass
```

Here, `Invoice` only handles calculation, and `InvoicePrinter` only handles printing. Each class has exactly one reason to ever change — making the codebase far easier to maintain and test.

3.2 Open/Closed Principle (OCP)

The Rule: Software entities should be open for extension, but closed for modification.

This means you should be able to add new behavior to your system WITHOUT rewriting existing, already-tested code. You extend by adding new code (like a new subclass), not by editing old code and risking new bugs.

```
class Shape:
    def area(self):
        pass

# To add a new shape, you create a NEW subclass of Shape.
# You never have to touch the existing Shape class.
```

3.3 Liskov Substitution Principle (LSP)

The Rule: Objects of a derived (child) class must be substitutable for objects of the base (parent) class, without breaking the program.

In other words, if your code expects a `Bird` object, ANY subclass of `Bird` should be able to stand in its place and behave sensibly. If a subclass breaks the expectations set by its parent, you've violated LSP.

```
class Bird:
    def fly(self):
        pass

# A classic LSP trap: what if you create a Penguin(Bird)
# subclass? Penguins can't fly, so calling fly() on a
# Penguin would break the contract the Bird class promised.
# A better design might separate "FlyingBird" from "Bird".
```

⚠ Common Trap: LSP violations often hide in seemingly natural inheritance hierarchies. Just because a Penguin 'is-a' Bird in biology doesn't mean it should inherit `fly()` in your code if not all birds can fly.

3.4 Interface Segregation Principle (ISP)

The Rule: Clients should not be forced to depend on methods they don't use.

Instead of one giant, bloated interface with a dozen methods, prefer several small, focused interfaces. A class that only needs to print shouldn't be forced to also implement `scan()` and `fax()` methods just because they all live under one giant "OfficeMachine" interface.

```
class Printer:
    def print(self):
        pass

# If you also need scanning, create a SEPARATE
# Scanner interface, rather than bloating Printer.
```

3.5 Dependency Inversion Principle (DIP)

The Rule: High-level modules should not depend on low-level modules directly. Both should depend on abstractions.

This is the principle that makes your code testable and flexible. Instead of your business logic directly depending on a specific concrete database class, it should depend on an abstract "Database" interface. The concrete implementation can then be swapped freely — for testing, for switching vendors, for anything — without touching your business logic.

```
class Database:
    def save(self):
        pass

# Your business logic depends on the Database ABSTRACTION,
# not on a specific concrete database like PostgreSQLDatabase
# or MongoDBDatabase. This supports dependency injection.
```

Chapter 4: Interfaces in Python — Abstract Base Classes

4.1 Why Python Needs This

Many languages (like Java or C#) have a built-in

interface keyword that lets you define a contract — a set of methods a class **MUST** implement, without specifying how. Python doesn't have a dedicated keyword, but it provides the same capability through

Abstract Base Classes (ABCs), available via the built-in `abc` module.

ABCs were formally introduced through

PEP 3119 and have been available since Python 2.6 and Python 3.0. They let you:

- Define abstract methods that subclasses **MUST** implement.
- Prevent a class from being instantiated directly if it has unimplemented abstract methods.
- Enforce the Interface Segregation Principle by acting as small, focused contracts.

4.2 Defining an Abstract Base Class

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
```

Here, `Shape` inherits from `ABC`, marking it as an abstract class. The `@abstractmethod` decorator on `area()` tells Python: "Any class that inherits from `Shape` **MUST** override and implement this method, or Python will raise an error when you try to instantiate it."

4.3 Implementing a Concrete Class

A class that inherits from an ABC and implements **ALL** of its abstract methods is called a **concrete class**. Only concrete classes can be instantiated.

```
class Circle(Shape):
    def area(self):
        return 3.14 * 5 * 5

c = Circle()
print(c.area())    # 78.5
```

 **What If You Forget?** If you create a class that inherits from `Shape` but forgets to implement `area()`, Python will raise a `TypeError` the moment you try to instantiate it — not when you call `area()`, but the instant you write `Circle()`. This 'fail fast' behavior catches bugs early.

Chapter 5: The Builder Pattern (Creational)

5.1 The Problem It Solves

Picture a class with a constructor that takes 15 parameters — some optional, some required, in a very specific order. Calling it becomes error-prone and unreadable:

`Computer(None, None, True, False, None, "16GB", ...)`. What does each value even mean without checking the constructor signature every time?

The

Builder Pattern solves this by separating the construction of a complex object from its final representation. Instead of one giant constructor call, you build the object step-by-step using clearly named methods.

5.2 Structure

- Client — creates a Director (or directly uses a Builder).
- Director — controls the construction steps and their order (optional in simpler implementations).
- Builder — defines the methods used to construct each piece of the object.
- Concrete Builder — implements those construction methods.
- Product — the final object, returned to the client once construction is complete.

5.3 Benefits

- Reduces constructor complexity — no more giant parameter lists.
- Hides internal implementation details of how the object is assembled.
- Enforces a specific construction order when one is required.
- Supports building multiple different representations of similar objects.
- Separates "how something is built" from "what it ultimately becomes".

 **Real-World Analogy:** Ordering a custom burger isn't done by shouting all 15 ingredients at once. You build it step by step: choose the bun, add the patty, add toppings, add sauce. That's exactly what the Builder pattern models in code.

Chapter 6: The Prototype Pattern (Creational)

6.1 The Problem It Solves

Sometimes, creating a brand-new object from scratch is expensive — maybe it requires a database call, complex computation, or loading a large file. If you already have a similar object sitting around, why not just

clone it instead of building a new one from zero? That's exactly what the

Prototype Pattern does — it creates new objects by copying ('cloning') an existing prototype instance, rather than instantiating a class directly.

6.2 Benefits

- Faster than repeatedly building objects from scratch.
- Simplifies object creation logic significantly.
- Supports runtime object creation and inheritance.
- Allows you to customize a clone after copying it, without affecting the original.
- Reduces the total number of classes your system needs.

6.3 Shallow Copy vs. Deep Copy

Python's built-in `copy` module gives you two flavors of cloning, and the difference matters a lot once your objects contain nested structures (like lists inside lists, or objects inside objects).

Aspect	Shallow Copy	Deep Copy
What it copies	Top-level object; nested objects are SHARED (by reference)	Recursively copies EVERY nested object — fully independent
Speed	Faster	Slower
Memory usage	Lower	Higher
Side effects	Changing a nested object affects BOTH copies	Changes to one copy never affect the other
Best for	Simple, flat structures	Complex, nested structures requiring true independence

```
import copy

# Shallow Copy - nested list [2, 3] is SHARED between both
original = [1, [2, 3]]
clone = copy.copy(original)

# Deep Copy - nested list [2, 3] is FULLY duplicated
original = [1, [2, 3]]
clone = copy.deepcopy(original)
```

 **The Classic Bug:** If you shallow-copy an object containing a nested list and then modify that nested list through the CLONE, you will accidentally modify the ORIGINAL

too — because they share the same reference to the nested list. Always reach for `deepcopy()` when full independence matters.

6.4 The Prototype Manager

In larger systems, it's common to introduce a

Prototype Manager — a centralized registry that stores reusable prototype instances and hands out clones whenever they're needed. This centralizes the cloning logic and simplifies object management across a large codebase.

Chapter 7: The Adapter Pattern (Structural)

7.1 The Problem It Solves

Imagine your code expects objects with a `get_address()` method, but a third-party library you just integrated exposes a method called `fetch_location()` instead. You can't (or shouldn't) rewrite the third-party library — and you don't want to rewrite your existing code either.

The

Adapter Pattern (sometimes called the

Wrapper Pattern) solves exactly this — it converts one interface into another, allowing two otherwise-incompatible classes to work together seamlessly, without modifying either one.

 **Real-World Analogy:** A travel power plug adapter doesn't change your laptop charger or the wall socket — it simply sits between them, translating one shape into the other so both can work together.

7.2 Object Adapter (Composition-Based)

This approach uses

composition — the adapter "wraps" an instance of the adaptee and delegates calls to it internally. This is generally the

preferred approach because it works with the adaptee AND all of its subclasses, and avoids the rigidity of inheritance.

```
class Vendor:
    def get_address(self):
        return "123 Main St"

class VendorAdapter:
    def __init__(self, vendor):
        self.vendor = vendor

    @property
```

```
def address(self):  
    return self.vendor.get_address()
```

7.3 Class Adapter (Inheritance-Based)

This approach uses

inheritance — the adapter extends the adaptee class directly and overrides methods as needed. It's simpler to write but

less flexible because it only works with one specific adaptee class, not its variations.

```
class VendorAdapter(Vendor):  
    @property  
    def address(self):  
        return self.get_address()
```

7.4 Advantages

- Reuses existing, already-tested code instead of rewriting it.
- Makes it easy to integrate third-party libraries with mismatched interfaces.
- Reduces duplicate code across your codebase.
- Supports the Open/Closed Principle — no need to modify existing classes.

Chapter 8: The Proxy Pattern (Structural)

8.1 The Problem It Solves

Sometimes you don't want code to access an object directly — you want to add a layer of control in between, without changing the object itself or how callers interact with it. The

Proxy Pattern provides exactly that: an object that acts as a placeholder or stand-in for a "real" object, exposing the exact same interface but adding extra behavior — like access control, lazy loading, or logging — transparently.

8.2 The Four Common Proxy Types

Proxy Type	Purpose
Protection Proxy	Controls access permissions before allowing a method call
Virtual Proxy	Delays expensive object creation until it's actually needed
Remote Proxy	Hides the complexity of communicating with a remote object
Smart Proxy	Adds extra housekeeping logic (e.g., logging, reference counting)

8.3 Protection Proxy Example

A Protection Proxy verifies permissions before allowing the underlying operation to execute — extremely common in enterprise and business systems where access control is critical.

```
class Employee:
    def get_salary(self):
        return 100000

class EmployeeProxy:
    def __init__(self, employee, is_admin):
        self.employee = employee
        self.is_admin = is_admin

    def get_salary(self):
        if self.is_admin:
            return self.employee.get_salary()
        return "Access Denied"
```

Notice that `EmployeeProxy` exposes the exact same method name — `get_salary()` — as the real `Employee` class. Calling code doesn't even need to know it's talking to a proxy; the access-control check happens transparently behind the scenes.

8.4 Virtual Proxy Example

A Virtual Proxy delays the creation of an expensive object until it's genuinely needed — a technique often called

lazy loading. In Python, this is frequently implemented using caching decorators like `lru_cache`.

```
from functools import lru_cache

@lru_cache()
def load_data():
    return "Large Dataset"
```

The first call to `load_data()` actually performs the (presumably expensive) work. Every subsequent call returns the cached result instantly instead of redoing the work — improving startup performance and conserving resources.

8.5 Advantages

- Controls access to the real object transparently, without callers noticing.
- Follows the Open/Closed Principle — no changes needed to the real subject.
- Adds new functionality (security, caching, logging) without touching original code.
- Multiple proxy types can be layered together for combined behavior.

Chapter 9: The Strategy Pattern (Behavioral)

9.1 The Core Idea

The

Strategy Pattern (also called the

Policy Pattern) defines a family of interchangeable algorithms, encapsulates each one separately, and makes them swappable at runtime — without the surrounding code needing to know which specific algorithm is currently in use.

9.2 The Motivating Problem: A Shipping Cost Calculator

Imagine an e-commerce application that needs to calculate shipping costs for three carriers: FedEx, UPS, and Postal Service. Each carrier calculates cost using a completely different formula, and new carriers may be added later. The system needs to stay easy to extend.

The Naive (Bad) Approach

```
class Order:
    def calculate_shipping(self, shipper):
        if shipper == "UPS":
            return 10
        elif shipper == "FedEx":
            return 15
        elif shipper == "Postal":
            return 5
```

This looks innocent at first, but it quietly violates THREE of the SOLID principles at once:

- Single Responsibility Principle — Order is now responsible for both order management AND shipping calculation logic.
- Open/Closed Principle — adding DHL as a new carrier means editing this method directly, risking new bugs in already-working code.
- Dependency Inversion Principle — Order depends directly on concrete shipping logic instead of an abstraction.

Beyond the SOLID violations, this if/elif/else chain is also a recognized

design smell:

- It's difficult to maintain as more carriers are added.
- Each algorithm can't be tested in isolation.
- Every new requirement forces a modification to existing, working code.
- It increases coupling between unrelated concerns.

9.3 The Strategy Pattern Structure

- Context — holds the core business logic and delegates the actual algorithm work to a strategy object.
- Strategy (interface/ABC) — defines the common contract every algorithm must follow.
- Concrete Strategy — implements one specific algorithm, fully independent of the others.
- Client — selects the desired strategy and injects it into the Context.

9.4 Defining the Strategy Interface

As we learned in Chapter 4, Python uses Abstract Base Classes to define interfaces. Here, every shipping strategy will follow the same contract.

```
from abc import ABC, abstractmethod

class ShippingStrategy(ABC):
    @abstractmethod
    def calculate(self, weight):
        pass
```

9.5 Implementing Concrete Strategies

Each shipping carrier becomes its own small, independent, testable class — implementing the same `calculate()` method, but with completely different internal logic.

```
class UPSStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 2

class FedExStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 3

class PostalStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 1.5
```

Notice that none of these classes know about each other, and none of them know about `Order` either. They are fully isolated — each one can be tested completely on its own.

9.6 The Context Class

The `Order` class (our Context) no longer contains ANY shipping-specific logic. It simply holds a reference to whichever strategy was injected, and delegates the actual calculation to it.

```
class Order:
    def __init__(self, strategy):
        self.strategy = strategy

    def shipping_cost(self, weight):
        return self.strategy.calculate(weight)
```

9.7 Putting It All Together

```
order = Order(UPSStrategy())
print(order.shipping_cost(10))

# Output: 20
```

Want to switch carriers? Just inject a different strategy — `Order` never has to change:

```
order = Order(FedExStrategy())
print(order.shipping_cost(10))
```

```
# Output: 30
```

9.8 How Strategy Fixes the SOLID Violations

Principle	How Strategy Satisfies It
Single Responsibility	Order handles orders only; each strategy handles its own shipping math only.
Open/Closed	Adding a new carrier (e.g., DHL) means writing a new class — never editing Order or other strategies.
Dependency Inversion	Order depends on the ShippingStrategy abstraction, never on a concrete carrier class.

Adding a new carrier later is now trivial and completely safe:

```
class DHLStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 4

# No existing code was touched to add this!
```

9.9 Bonus: Testing Made Easy

Because strategies are isolated and injected, you can easily swap in a mock strategy during unit tests — without touching any real shipping logic.

```
class MockStrategy(ShippingStrategy):
    def calculate(self, weight):
        return 109

# Inject MockStrategy() in tests to get predictable,
# repeatable results — no real carrier logic involved.
```

9.10 Lighter-Weight Strategy Implementations

Full classes aren't always necessary. For simple algorithms, Python lets you use plain functions or even lambda expressions as strategies — reducing boilerplate considerably.

Strategies as Functions

```
def ups_shipping(weight):
    return weight * 2

class Order:
    def __init__(self, strategy):
        self.strategy = strategy
```

```
def shipping_cost(self, weight):
    return self.strategy(weight)

order = Order(ups_shipping)
```

Strategies as Lambda Expressions

```
ups_strategy = lambda weight: weight * 2
print(ups_strategy(10))

# Output: 20
```

 **When to Use Which:** Use full classes when an algorithm needs internal state or multiple methods. Use plain functions when the algorithm is simple and stateless. Use lambdas only for very short, throwaway calculations — they get unreadable fast if overused.

9.11 When to Use (and Avoid) the Strategy Pattern

Use Strategy When...	Avoid Strategy When...
Multiple algorithms solve the same problem	Only one algorithm will ever exist
Algorithms change or get added frequently	Algorithms rarely or never change
You see long if/elif/else chains forming	The added abstraction creates unnecessary complexity
Runtime algorithm selection is required	The project is very small / simple solutions suffice
SOLID compliance matters for this code	—

9.12 Red Flags That Suggest You Need This Pattern

- Long if/elif/else chains selecting between behaviors.
- Repeated algorithm-selection logic scattered across the codebase.
- Frequent code modifications every time new behavior is required.
- Tight coupling between algorithms and core business logic.
- Difficulty testing individual algorithms in isolation.

SECTION 2

Practical & Coding — IPython Notebook Walkthroughs

Let's bring every pattern from Section 1 to life. Each notebook below is broken into cells, with every single line explained in detail — exactly as you'd want to document it for your own reference book.

Notebook 1: SOLID Principles & Abstract Base Classes

Cell 1 — Single Responsibility Principle in Action

```
class Invoice:
    def calculate_total(self, items):
        return sum(items)

class InvoicePrinter:
    def print_invoice(self, total):
        print(f"Invoice Total: ${total}")

# Usage
invoice = Invoice()
total = invoice.calculate_total([100, 250, 75])

printer = InvoicePrinter()
printer.print_invoice(total)
```

Code Line	What It Does
<code>class Invoice:</code>	Defines a class whose ONLY job is invoice-related math. It knows nothing about printing, emailing, or formatting — just calculation. This is SRP in practice.
<code>def calculate_total(self, items):</code>	A method that accepts a list of item prices and returns their sum. The 'self' parameter refers to the current Invoice instance, as is standard in all Python instance methods.
<code>return sum(items)</code>	Python's built-in sum() function adds up every number in the list. This is the entire calculation logic — simple and isolated.
<code>class InvoicePrinter:</code>	A completely SEPARATE class dedicated only to display/output logic. If tomorrow we need to change HOW invoices are printed (maybe formatted differently, or sent to a printer API), we only touch this class — Invoice remains untouched.
<code>print(f"Invoice Total: \${total}")</code>	An f-string (formatted string literal) that embeds the 'total' variable directly into the printed text. The f prefix before the quotes lets Python evaluate {total} inline.
<code>invoice = Invoice()</code>	Creates an instance of the Invoice class. At this point, no calculation has happened yet — we just have an empty Invoice object ready to use.

<code>total = invoice.calculate_total([100, 250, 75])</code>	Calls <code>calculate_total()</code> on our invoice instance, passing a list of three prices. The method returns 425 (100+250+75), which gets stored in 'total'.
<code>printer = InvoicePrinter()</code>	Creates an instance of our separate printer class.
<code>printer.print_invoice(total)</code>	Passes the calculated total to the printer, which formats and displays it. Notice how Invoice and InvoicePrinter never directly reference each other's internals — they're loosely coupled.

Cell 2 — Abstract Base Class: Defining a Contract

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimeter(self):
        pass
```

Code Line	What It Does
<code>from abc import ABC, abstractmethod</code>	Imports two things from Python's built-in abc module: ABC, the base class your abstract classes must inherit from, and abstractmethod, a decorator that marks a method as required-to-implement.
<code>class Shape(ABC):</code>	Defines Shape as an abstract class by inheriting from ABC. This single change makes Shape uninstaniable on its own — you can never write Shape() directly; Python will raise a TypeError.
<code>@abstractmethod\ndef area(self):</code>	Marks area() as a method every subclass MUST override. The 'pass' body means this method does nothing by itself — it exists purely as a placeholder defining the contract's shape.
<code>@abstractmethod\ndef perimeter(self):</code>	A second required method. Any subclass of Shape must implement BOTH area() and perimeter(), or it too will be considered abstract and uninstaniable.

Cell 3 — Implementing Concrete Classes

```
class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14159 * self.radius ** 2
```

```

    def perimeter(self):
        return 2 * 3.14159 * self.radius

c = Circle(5)
print(c.area())          # 78.53975
print(c.perimeter())     # 31.4159

```

Code Line	What It Does
<code>class Circle(Shape):</code>	Circle inherits from Shape. Because it's going to implement BOTH abstract methods, it becomes a concrete (instantiable) class.
<code>def __init__(self, radius):</code>	The constructor. Stores the radius value passed in when creating a Circle instance, so it can be reused across multiple methods.
<code>self.radius ** 2</code>	The <code>**</code> operator means 'raised to the power of'. <code>self.radius ** 2</code> squares the radius value — needed for the area formula πr^2 .
<code>c = Circle(5)</code>	Creates a Circle instance with radius=5. Because Circle implements every abstract method from Shape, this line works without errors.
<code>c.area()</code>	Calls the <code>area()</code> method we defined, returning $3.14159 * 25 = 78.53975$.

Cell 4 — What Happens If You Forget a Method?

```

class Square(Shape):
    def __init__(self, side):
        self.side = side

    def area(self):
        return self.side ** 2

    # perimeter() is MISSING!

s = Square(4)    # Raises TypeError!

```

Code Line	What It Does
<code>class Square(Shape):</code>	Inherits from Shape, but only implements <code>area()</code> — <code>perimeter()</code> is never overridden.
<code>s = Square(4)</code>	This line FAILS with: <code>TypeError: Can't instantiate abstract class Square with abstract method perimeter.</code> Python catches the missing implementation at the moment of instantiation — long before you'd ever call <code>perimeter()</code> and discover the bug at runtime. This 'fail-fast' behavior is one of the biggest benefits of using ABCs.

Notebook 2: The Builder Pattern

Cell 1 — Defining the Product

```
class Computer:
    def __init__(self):
        self.cpu = None
        self.ram = None
        self.storage = None

    def __str__(self):
        return f"Computer(cpu={self.cpu}, ram={self.ram},
storage={self.storage})"
```

Code Line	What It Does
class Computer:	This is the PRODUCT — the final object we're trying to construct step-by-step. Notice it has a very simple constructor that just initializes everything to None.
self.cpu = None / self.ram = None	Each attribute starts empty. The Builder (defined next) will be responsible for filling these in piece by piece, rather than passing everything through one giant constructor call.
def __str__(self):	A special Python 'dunder' method that controls what gets printed when you call print() on a Computer instance. Without this, print(computer) would show an unhelpful memory address instead of readable data.

Cell 2 — Building the ComputerBuilder

```
class ComputerBuilder:
    def __init__(self):
        self.computer = Computer()

    def add_cpu(self, cpu):
        self.computer.cpu = cpu
        return self

    def add_ram(self, ram):
        self.computer.ram = ram
        return self

    def add_storage(self, storage):
        self.computer.storage = storage
        return self

    def build(self):
        return self.computer
```

Code Line	What It Does
<code>self.computer = Computer()</code>	The moment a ComputerBuilder is created, it immediately creates an empty Computer instance internally, ready to be filled in step by step.
<code>def add_cpu(self, cpu):</code>	A 'setter-style' method that sets the cpu attribute on the internal Computer instance. Notice it takes the actual cpu value as a parameter now — making the builder reusable for any CPU, not hardcoded.
<code>return self</code>	This is the secret behind method chaining! By returning 'self' (the builder instance itself), you can chain multiple calls together: builder.add_cpu('Intel').add_ram('16GB')... Without 'return self', each method call would return None and chaining would break.
<code>def build(self):</code>	The final step — returns the fully assembled Computer object to the caller. This signals 'construction is complete, here's your product'.

Cell 3 — Using the Builder

```
builder = ComputerBuilder()

gaming_pc = (
    builder
    .add_cpu("Intel i9")
    .add_ram("32GB")
    .add_storage("2TB SSD")
    .build()
)

print(gaming_pc)

# Output: Computer(cpu=Intel i9, ram=32GB, storage=2TB SSD)
```

Code Line	What It Does
<code>builder = ComputerBuilder()</code>	Creates a new builder instance. Internally, this already created an empty Computer object, ready to be configured.
<code>.add_cpu("Intel i9")\n.add_ram("32GB")\n.add_storage("2TB SSD")</code>	Each chained call configures one attribute and returns the builder itself, enabling the NEXT call to chain off it immediately. This reads almost like a sentence: 'add a CPU, then add RAM, then add storage'.
<code>.build()</code>	Finalizes the process and hands back the fully assembled Computer object. From here, the builder has done its job —

	gaming_pc now holds the complete product.
<code>print(gaming_pc)</code>	Triggers the <code>__str__</code> method we defined on Computer, producing a clean, readable summary of the final configuration.

 **Compare the Alternative:** Without a Builder, you'd write something like `Computer('Intel i9', '32GB', '2TB SSD', None, None, True)` — a confusing wall of positional arguments. The Builder's chained, named method calls are far more self-documenting.

Notebook 3: The Prototype Pattern

Cell 1 — Shallow Copy Behavior

```
import copy

original = [1, [2, 3]]
clone = copy.copy(original)

clone[0] = 99          # Change top-level item
clone[1].append(4)     # Change NESTED item

print("Original:", original)
print("Clone:", clone)
```

Code Line	What It Does
<code>import copy</code>	Loads Python's built-in copy module, which provides both <code>copy()</code> (shallow) and <code>deepcopy()</code> (deep) functions.
<code>original = [1, [2, 3]]</code>	A list containing an integer (1) and a NESTED list ([2, 3]). This nested structure is exactly where shallow vs. deep copy behavior diverges.
<code>clone = copy.copy(original)</code>	Creates a shallow copy. A new outer list object is created, but the nested list [2, 3] inside it is NOT duplicated — both 'original' and 'clone' point to the SAME nested list object in memory.
<code>clone[0] = 99</code>	Changes the top-level first element of clone only. Since integers are immutable and this is a top-level slot, 'original' is unaffected — <code>original[0]</code> remains 1.
<code>clone[1].append(4)</code>	Appends 4 to the NESTED list. Because <code>clone[1]</code> and <code>original[1]</code> are literally the SAME list object (shared reference from the shallow copy), this change is visible in BOTH original and clone!

<code>print("Original:", original)</code>	Will print: Original: [1, [2, 3, 4]] — notice the nested list changed even though we only modified 'clone'. This is the shallow copy 'gotcha' in action.
---	--

Cell 2 — Deep Copy Behavior (The Fix)

```
import copy

original = [1, [2, 3]]
clone = copy.deepcopy(original)

clone[0] = 99
clone[1].append(4)

print("Original:", original)
print("Clone:", clone)
```

Code Line	What It Does
<code>clone = copy.deepcopy(original)</code>	Creates a deep copy. This time, Python recursively walks through the ENTIRE structure and creates brand-new, independent copies of every nested object — including the inner [2, 3] list.
<code>clone[1].append(4)</code>	Appends 4 to clone's nested list. Because deepcopy() gave clone its OWN independent nested list, this change does NOT affect 'original' at all.
<code>print("Original:", original)</code>	Will print: Original: [1, [2, 3]] — completely unchanged. This proves the two objects are now fully independent of each other.

Cell 3 — A Practical Prototype: Cloning a Configuration Object

```
import copy

class ServerConfig:
    def __init__(self, cpu, ram, tags):
        self.cpu = cpu
        self.ram = ram
        self.tags = tags

    def clone(self):
        return copy.deepcopy(self)

    def __str__(self):
        return f"ServerConfig(cpu={self.cpu}, ram={self.ram}, tags={self.tags})"

base_config = ServerConfig("4 vCPU", "16GB", ["prod"])

staging_config = base_config.clone()
```

```

staging_config.tags.append("staging")

print(base_config)
print(staging_config)

```

Code Line	What It Does
<code>def clone(self):</code>	This is the heart of the Prototype Pattern — a method living right on the class itself, letting any <code>ServerConfig</code> instance produce an independent copy of itself on demand.
<code>return copy.deepcopy(self)</code>	Uses <code>deepcopy()</code> (not <code>copy()</code> !) deliberately. Since this object holds a 'tags' list, we want the clone's tags list to be fully independent — otherwise modifying <code>staging_config.tags</code> would also silently corrupt <code>base_config.tags</code> .
<code>base_config = ServerConfig(...)</code>	Creates our 'prototype' instance — the template configuration we want to base new ones on, instead of rebuilding from scratch each time.
<code>staging_config = base_config.clone()</code>	Instead of constructing a brand new <code>ServerConfig</code> from zero, we clone the existing prototype. This is faster and guarantees <code>staging_config</code> starts with identical values to <code>base_config</code> .
<code>staging_config.tags.append("staging")</code>	Modifies ONLY the clone's tags list. Because we used <code>deepcopy()</code> , <code>base_config.tags</code> remains ['prod'] — completely unaffected by this change.

Notebook 4: The Adapter Pattern

Cell 1 — The Incompatible Interfaces Problem

```

# Third-party vendor library (you can't modify this code)
class Vendor:
    def get_address(self):
        return "123 Main St"

# Your existing system expects objects with an 'address' PROPERTY,
# not a get_address() METHOD
def print_shipping_label(obj):
    print(f"Ship to: {obj.address}")

```

Code Line	What It Does
<code>class Vendor:</code>	Represents a third-party class — imagine this comes from an external pip package you have no control over. It exposes <code>get_address()</code> as a METHOD.

<code>def print_shipping_label(obj):</code>	Your existing, already-working function. It expects whatever object it receives to have an 'address' attribute — accessed WITHOUT parentheses, like a property, not a method call.
<code>print(f"Ship to: {obj.address}")</code>	If you passed a raw Vendor instance here directly, this would FAIL — Vendor has <code>get_address()</code> , not a plain 'address' property. This mismatch is exactly the problem the Adapter Pattern exists to solve.

Cell 2 — Building an Object Adapter

```
class VendorAdapter:
    def __init__(self, vendor):
        self.vendor = vendor

    @property
    def address(self):
        return self.vendor.get_address()
```

Code Line	What It Does
<code>def __init__(self, vendor):</code>	The adapter's constructor accepts an existing Vendor instance and stores it. This is the COMPOSITION part of 'Object Adapter' — we wrap the original object rather than inheriting from it.
<code>@property</code>	This decorator turns the <code>address()</code> method into something you can access WITHOUT parentheses — exactly like a regular attribute. This is the trick that makes VendorAdapter look like it has a real 'address' property.
<code>return self.vendor.get_address()</code>	Internally, the adapter delegates the actual work to the wrapped vendor object's <code>get_address()</code> method. The caller never needs to know this translation is happening.

Cell 3 — Using the Adapter Seamlessly

```
vendor = Vendor()
adapted_vendor = VendorAdapter(vendor)

print_shipping_label(adapted_vendor)

# Output: Ship to: 123 Main St
```

Code Line	What It Does
<code>vendor = Vendor()</code>	Creates the original, unmodified third-party object.

<code>adapted_vendor = VendorAdapter(vendor)</code>	Wraps it in our adapter. Now <code>adapted_vendor</code> exposes an <code>'address'</code> property even though the original <code>Vendor</code> class never had one.
<code>print_shipping_label(adapted_vendor)</code>	Our existing function, written with zero knowledge of <code>Vendor</code> , works perfectly because <code>adapted_vendor.address</code> quietly calls <code>vendor.get_address()</code> under the hood. Neither <code>Vendor</code> nor <code>print_shipping_label</code> had to change a single line.

Notebook 5: The Proxy Pattern

Cell 1 — Protection Proxy: Access Control

```
class Employee:
    def get_salary(self):
        return 100000

class EmployeeProxy:
    def __init__(self, employee, is_admin):
        self.employee = employee
        self.is_admin = is_admin

    def get_salary(self):
        if self.is_admin:
            return self.employee.get_salary()
        return "Access Denied"

real_employee = Employee()

admin_view = EmployeeProxy(real_employee, is_admin=True)
guest_view = EmployeeProxy(real_employee, is_admin=False)

print(admin_view.get_salary())    # 100000
print(guest_view.get_salary())    # Access Denied
```

Code Line	What It Does
<code>class Employee:</code>	The REAL object — the thing we ultimately want to protect access to. It has no security logic of its own; it just does its job.
<code>class EmployeeProxy:</code>	The proxy class. It exposes the EXACT same method name (<code>get_salary</code>) as <code>Employee</code> , so it's a drop-in, transparent replacement from the caller's perspective.
<code>self.is_admin = is_admin</code>	Stores a boolean flag indicating whether the current user viewing this proxy has admin privileges. This is the access-control state the proxy is responsible for.

<code>if self.is_admin: return self.employee.get_salary()</code>	Only forwards the call to the real Employee object if the admin flag is True. This is the actual 'protection' logic — the real object is never even consulted otherwise.
<code>return "Access Denied"</code>	If the check fails, the proxy short-circuits and returns a rejection message instead of ever touching the real, sensitive data.
<code>admin_view = EmployeeProxy(real_employee, is_admin=True)</code>	Wraps the SAME real_employee object, but configured for an admin user. Both admin_view and guest_view wrap the identical underlying Employee — only the access rules differ.
<code>guest_view.get_salary()</code>	Returns 'Access Denied' because is_admin=False. Notice the guest never even gets close to seeing the real salary figure — full protection.

Cell 2 — Virtual Proxy: Lazy Loading with Caching

```
from functools import lru_cache
import time

@lru_cache()
def load_data():
    print("Loading large dataset... (expensive operation)")
    time.sleep(2)    # Simulate a slow operation
    return "Large Dataset"

print("First call:")
print(load_data())

print("Second call:")
print(load_data())
```

Code Line	What It Does
<code>from functools import lru_cache</code>	Imports Python's built-in Least Recently Used cache decorator. It's a clean, built-in way to implement Virtual Proxy-style lazy loading and caching without writing your own caching logic.
<code>import time</code>	Imported only to simulate a slow operation using <code>time.sleep()</code> — in a real project, this expensive step might be a database query, an API call, or loading a massive file from disk.
<code>@lru_cache()</code>	Decorates <code>load_data()</code> . The FIRST time this function is called with a given set of arguments, Python runs it normally and CACHES the result. Every SUBSEQUENT call with the same arguments returns the cached result instantly, without re-running the function body.

<code>time.sleep(2)</code>	Pauses execution for 2 seconds, simulating an expensive, slow operation — like fetching data over a network.
<code>print(load_data())</code> <code># First call</code>	This call actually executes the function body, prints the 'Loading...' message, waits 2 seconds, then returns and prints 'Large Dataset'.
<code>print(load_data())</code> <code># Second call</code>	This call does NOT print 'Loading...' and does NOT wait 2 seconds — it instantly returns the cached value from the first call. This is the Virtual Proxy pattern in action: the expensive object/result is only truly created once, then reused.

Notebook 6: The Strategy Pattern

Cell 1 — Defining the Strategy Interface and Concrete Strategies

```
from abc import ABC, abstractmethod

class ShippingStrategy(ABC):
    @abstractmethod
    def calculate(self, weight):
        pass

class UPSStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 2

class FedExStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 3

class PostalStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 1.5
```

Code Line	What It Does
<code>class ShippingStrategy(ABC):</code>	Our abstract interface — the contract every shipping algorithm must satisfy. Defining this upfront guarantees that Order (our Context, defined next) can rely on ANY strategy having a <code>calculate()</code> method, regardless of carrier.
<code>class UPSStrategy(ShippingStrategy):</code>	A concrete strategy implementing UPS-specific pricing logic: \$2 per unit of weight. Completely isolated from FedEx and Postal logic — change this class, and nothing else breaks.
<code>class FedExStrategy(...)</code> / <code>class PostalStrategy(...)</code>	Two more concrete strategies, each with their own pricing formula (\$3/unit and \$1.5/unit respectively). All

	three classes are interchangeable from the Context's point of view because they share the same interface.
--	---

Cell 2 — The Context Class

```
class Order:
    def __init__(self, strategy):
        self.strategy = strategy

    def shipping_cost(self, weight):
        return self.strategy.calculate(weight)
```

Code Line	What It Does
<code>def __init__(self, strategy):</code>	The Order constructor accepts a 'strategy' object — this is dependency injection in its simplest form. Order doesn't create the strategy itself; it's handed one from outside, satisfying the Dependency Inversion Principle.
<code>self.strategy = strategy</code>	Stores the injected strategy as an instance attribute, ready to be used whenever <code>shipping_cost()</code> is called.
<code>return self.strategy.calculate(weight)</code>	Delegates ALL of the actual calculation work to whichever strategy was injected. Order itself contains zero shipping-math logic — it only knows HOW to ask for the cost, not HOW the cost is computed.

Cell 3 — Running and Comparing All Strategies

```
weight = 10

strategies = {
    "UPS": UPSStrategy(),
    "FedEx": FedExStrategy(),
    "Postal": PostalStrategy(),
}

for name, strategy in strategies.items():
    order = Order(strategy)
    cost = order.shipping_cost(weight)
    print(f"{name}: ${cost}")
```

Code Line	What It Does
<code>weight = 10</code>	A fixed shipment weight we'll use to compare costs fairly across all three carriers.
<code>strategies = { ... }</code>	A dictionary mapping carrier names to their corresponding strategy INSTANCES (note the parentheses — these are already-created objects, not just class references).

<code>for name, strategy in strategies.items():</code>	Iterates over the dictionary. <code>.items()</code> gives us both the key (carrier name) and value (strategy instance) on each loop iteration.
<code>order = Order(strategy)</code>	Creates a NEW Order for each iteration, injecting a different strategy each time. This demonstrates the core power of the pattern — the SAME Order class produces different behavior purely based on which strategy it receives.
<code>cost = order.shipping_cost(weight)</code>	Calls our delegation method, which internally calls the specific strategy's <code>calculate()</code> method.
<code>print(f"{name}: \${cost}")</code>	Prints results: UPS: \$20, FedEx: \$30, Postal: \$15.0 — proving each strategy produces genuinely different output for identical input, exactly as designed.

Cell 4 — Extending with a New Carrier (Zero Modification)

```
class DHLStrategy(ShippingStrategy):
    def calculate(self, weight):
        return weight * 4

dhl_order = Order(DHLStrategy())
print("DHL:", dhl_order.shipping_cost(10))

# Output: DHL: 40
```

Code Line	What It Does
<code>class DHLStrategy(ShippingStrategy):</code>	A brand-new carrier added to the system. Notice we did NOT modify Order, UPSStrategy, FedExStrategy, or PostalStrategy at all — we simply added one new class. This is the Open/Closed Principle in its purest, most visible form.
<code>dhl_order = Order(DHLStrategy())</code>	Order accepts the new strategy without any code changes on its end — it was already designed to accept ANY object conforming to the ShippingStrategy interface.

Cell 5 — Strategies as Plain Functions (Lighter Alternative)

```
def ups_shipping(weight):
    return weight * 2

def fedex_shipping(weight):
    return weight * 3

class Order:
    def __init__(self, strategy):
        self.strategy = strategy
```

```

def shipping_cost(self, weight):
    return self.strategy(weight)    # Calling the function directly

order = Order(ups_shipping)
print(order.shipping_cost(10))

# Output: 20

```

Code Line	What It Does
<code>def ups_shipping(weight):</code>	Instead of a full class with an ABC and inheritance, this is just a plain function with the same signature pattern (takes weight, returns cost). Python's functions are first-class objects, so they can be passed around just like any other value.
<code>self.strategy(weight)</code>	Since 'strategy' is now a plain function rather than an object, we call it directly using parentheses — <code>strategy(weight)</code> — instead of <code>strategy.calculate(weight)</code> . The Context code adapts slightly, but the overall pattern (injectable, swappable behavior) is identical.
<code>order = Order(ups_shipping)</code>	Notice we pass the function itself — NOT <code>ups_shipping()</code> with parentheses, which would call it immediately. We pass the function OBJECT so Order can call it later, whenever <code>shipping_cost()</code> is invoked.

Cell 6 — Strategies as Lambda Expressions (Most Lightweight)

```

ups_strategy = lambda weight: weight * 2
fedex_strategy = lambda weight: weight * 3

order = Order(ups_strategy)
print(order.shipping_cost(10))

# Output: 20

```

Code Line	What It Does
<code>ups_strategy = lambda weight: weight * 2</code>	A lambda is an anonymous, inline function. 'lambda weight: weight * 2' is exactly equivalent to writing 'def f(weight): return weight * 2', just in a single compact expression. It's assigned to the variable <code>ups_strategy</code> for later use.
<code>order = Order(ups_strategy)</code>	Works exactly the same as passing a regular function — Order doesn't care HOW the strategy was defined, only that it's callable with one argument (weight).

 **Choosing the Right Implementation:** Use full classes (with ABC) when algorithms need internal state, multiple methods, or strict interface enforcement across a large team. Use plain functions for simple, stateless logic. Use lambdas only for the shortest, most throwaway calculations — readability suffers quickly if lambdas get too complex.

Quick Reference: Design Patterns Cheat Sheet

SOLID Principles at a Glance

Letter	Principle	One-Line Summary
S	Single Responsibility	One class, one job, one reason to change
O	Open/Closed	Extend with new code, don't modify existing code
L	Liskov Substitution	Subclasses must honor the parent's contract
I	Interface Segregation	Prefer small, focused interfaces over big bloated ones
D	Dependency Inversion	Depend on abstractions, not concrete implementations

Pattern Comparison at a Glance

Pattern	Category	Core Idea	Use When...
Builder	Creational	Construct objects step-by-step	Complex objects with many optional parts
Prototype	Creational	Clone existing objects instead of building anew	Object creation is expensive
Adapter	Structural	Translate one interface into another	Integrating incompatible/third-party code
Proxy	Structural	Control access to a real object	Need access control, lazy loading, or logging
Strategy	Behavioral	Swap interchangeable algorithms at runtime	Multiple algorithms solve the same problem

— End of Chapter —